UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias
Licenciatura en Ciencias de la Computación

Asignatura: Inteligencia Artificial

Laboratorio 2.13: Problemas de Satisfacción de Restricciones (CSP)
Tema: El Problema de las 8 Reinas

Profesora: Dra. Jessica Sarahi Méndez Rincón
Ayudante: Diego Eduardo Peña Villegas
Estudiante: Sita Zoé Sotomayor Carrillo
Fecha: 5 de marzo de 2026

1. Introducción

El problema de las 8 Reinas es un ejemplo clásico de un Problema de Satisfacción de Restricciones (CSP). El objetivo es colocar ocho reinas en un tablero de ajedrez estándar (8×8) de manera que ninguna pueda atacar a otra. Esto implica restricciones implícitas: ninguna reina puede compartir la misma fila, columna o diagonal. En este laboratorio se implementa una solución mediante **Backtracking** recursivo y se analiza su eficiencia.

2. Desarrollo

Se analizó el código base proporcionado, el cual utiliza un vector unidimensional `tablero` de tamaño 8, donde el índice representa la fila y el valor representa la columna. Esta estructura de datos es eficiente porque garantiza implícitamente que no haya dos reinas en la misma fila.

2.1. Funcionamiento del Algoritmo

La función `resolver_8_reinas(fila)` intenta colocar una reina en la fila actual iterando sobre las 8 columnas posibles.

1. Verifica si la posición es segura (sin conflictos con reinas anteriores).
2. Si es segura, coloca la reina y llama recursivamente a la función para la siguiente fila.
3. Si la llamada recursiva falla (retorna Falso), realiza *backtracking*: retira la reina y prueba la siguiente columna.

3. Análisis y Respuestas (Cuestionario)

3.1. 1. ¿Es un algoritmo óptimo?

El algoritmo de Backtracking **es completo**, ya que garantiza encontrar una solución si existe, pero **no es necesariamente óptimo en tiempo**.

- Explora el espacio de estados mediante fuerza bruta inteligente (poda ramas inválidas), pero en el peor de los casos sigue teniendo una complejidad exponencial.
- Para $N = 8$, es suficientemente rápido, pero para $N = 100$, este enfoque sería inviable sin heurísticas adicionales (como *Min-Conflicts*).

3.2. 2. Complejidad y Espacio de Búsqueda

- **Espacio de Estados:** En teoría, hay $\binom{64}{8}$ formas de poner 8 reinas, lo cual es enorme ($4{,}4 \times 10^9$). Sin embargo, al restringir una reina por fila, el espacio se reduce a 8^8 (16,7 millones).
- **Complejidad Temporal:** $O(N!)$. La primera reina tiene N opciones, la segunda $N-1$ (aprox), etc. El backtracking poda muchas ramas, pero la cota superior sigue siendo factorial.

- **Complejidad Espacial:** $O(N)$. Solo se necesita almacenar el arreglo del tablero y la pila de recursión, ambos de tamaño lineal respecto al número de reinas.

3.3. 3. Limitación del Código Actual

La principal limitación es que el código se detiene al encontrar **la primera solución**.

- El problema de las 8 Reinas tiene **92 soluciones distintas** (o 12 soluciones fundamentales si descontamos rotaciones y espejos).
- El algoritmo actual retorna `True` inmediatamente después de completar el tablero, ignorando el resto del espacio de búsqueda.

3.4. 4. Mejora Propuesta y Pseudocódigo

Para superar la limitación, se propone modificar el algoritmo para que almacene las soluciones en una lista en lugar de retornar.

Pseudocódigo Mejorado:

```
Procedimiento BuscarTodas(fila, tablero, lista_soluciones):
    Si fila == 8:
        Agregar COPIA(tablero) a lista_soluciones
        Retornar (No True, solo volver atras)

    Para columna de 0 a 7:
        Si EsSeguro(fila, columna):
            tablero[fila] = columna
            BuscarTodas(fila + 1, tablero, lista_soluciones)
            # Al volver, NO detenemos el bucle.
            # Seguimos probando columnas (Backtracking exhaustivo)
            tablero[fila] = -1
```

Esta mejora permite obtener el conjunto completo de las 92 soluciones, proporcionando una visión global del problema.

4. Conclusión

El algoritmo de Backtracking es una herramienta poderosa para problemas CSP como las N-Reinas. Aunque su complejidad temporal es elevada ($O(N!)$), la poda efectiva del árbol de búsqueda lo hace viable para N pequeños. La implementación optimizada para encontrar todas las soluciones permite comprender mejor la simetría y riqueza del problema.

5. Bibliografía

- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach.* Pearson.
- Bellman, R. (1962). Dynamic Programming Treatment of the Travelling Salesman Problem. *Journal of the ACM.*